COMPLETE ECOSYSTEM ARCHITECTURE

How Inaya Actually Works

Every contract, app, backend, and data flow across the ecosystem — written from the real code, not the pitch. For the founding team.

Document INAYA-ARCH-2026-V1 · Classification Internal · September 2026

SECTION 01

How To Read This Document

Inaya is one product with two connected engines: a decentralized storage/DePIN protocol (on-chain contracts, node operators, client-side encryption) and a set of applications built on top of it (the web dApp, Business Workspace SaaS, mobile app, two desktop apps, and three AI assistants). This document maps every piece, what it actually does, and how it connects to everything else.

Conventions used throughout:

- Everything here is on BNB Chain Testnet unless stated otherwise — nothing described is live on mainnet yet.
- Contract addresses given are real, deployed, and verified on BscScan as of this writing.
- Where a feature is deliberately unbuilt or deferred, it's labeled as such rather than glossed over — this is a working reference, not a pitch document.
- "Backend" always means the Next.js app at inaya-network-dapp — every mobile screen, desktop app, and the web dApp itself all call the same backend, there is no separate API server.

SECTION 02

The System At A Glance

REPO / APP	WHAT IT IS	RUNS ON
inaya-network-dapp	The backend (Next.js API routes + MongoDB) AND the main web dApp AND the Business Workspace web UI — one Next.js app serving all three.	Vercel, Node.js
contracts/ (repo root)	Solidity source for the protocol's own contracts (Staking, Vault, Escrow, Node Registry, Proof Registry, Security Layer). Deployed to BSC Testnet.	Solidity 0.8.20, Hardhat
custody-sdk	Client-side encryption/sharding library ("InayaKernel") — the code that actually encrypts and splits a file before it ever leaves a user's device.	JS, @noble crypto primitives, ethers v6
custody-sdk/packages/node-daemon	CLI a storage-node operator runs — registers on-chain, sends heartbeat telemetry. Published to npm.	Node.js CLI
inaya-mobile	React Native / Expo mobile app — a superset of the web dApp's consumer features plus Business Workspace, Learn, and Security.	Expo, React Native
inaya-desktop	Tauri (Rust) native wrapper around the Business Workspace web app.	Tauri, WebView2 (Win) / WebKitGTK (Linux)
inaya-dapp-desktop	Tauri native wrapper around the main dApp (faucet, staking, etc).	Tauri, same engines as above

> None of the desktop or mobile apps run their own backend logic beyond thin native chrome — every one of them calls the exact same inaya-network-dapp API routes the website does.

SECTION 03

The Two Engines

ENGINE 1 — DEPIN PROTOCOL

The foundation: users encrypt files client-side, shard them, and anchor ownership + integrity proofs on-chain. Independent node operators register capacity, earn $INAYA/USDT for storage and threat-reporting, and settle through timelocked, oracle-attested (not cryptographic) verification. This is the part that makes Inaya a real DePIN, not just a SaaS product with a token bolted on.

ENGINE 2 — APPLICATIONS

Everything a user actually touches: the web dApp, Business Workspace (SaaS document management for companies that will never think about blockchain), mobile app, two desktop wrappers, a public Security transparency page, an investor Data Room, and three Gemini-powered AI assistants. All of it sits on top of Engine 1's infrastructure, but most of it is reachable — and usable — without a wallet.

> **Why this split matters.** It's the reason a non-crypto business customer can use the Business Workspace with just an email address while a node operator, three layers down, is settling USDT through a timelocked escrow contract — the same infrastructure serves both, but almost nothing about Engine 1 leaks into Engine 2's user experience.

SECTION 04

On-Chain Layer — Contracts

10 contracts deployed and verified on BSC Testnet, split across two families that don't share any direct dependency on each other.

Core Protocol (6 contracts)

Custody, payments, staking, and node settlement.

CONTRACT	ADDRESS	JOB
InayaCustody ("Core Custody Contract")	0x7F5E6c...5a888	Asset-ownership ledger. batchRegisterAssets() writes fileHash → {owner, shard CIDs, size, timestamp}. This is the single source of truth for "who owns this file." Note: its Solidity source isn't in this repo — only its interface (IInayaCustody) is, referenced from InayaProofRegistry. It's deployed and live, just not tracked as source here yet.
Mock USDT (mUSDT)	0x6f16E2...45D	Testnet stand-in for USDT — every fee, stake, escrow, and invoice in the protocol is denominated in this token until mainnet.
$INAYA Token	0x3966a3...e94e	The real ERC-20 utility/governance token. Fixed supply, 30,000,000 tokens (per tokenomics).
InayaNodeRegistry	0xd12a38...FD881	Node operator registry + timelocked commission settlement. registerNode(capacityGB), tiered commission (30/40/50% by Entry/Mid/Enterprise), verifier-attested metrics, 36-hour settlement delay before anyone can call releaseSettlement().
RevenueRouter	0x76B0d4...3256	processCorporateInvoice(usdtAmount) — entry point for Corporate Reserve plan purchases. Source not in this repo, referenced by address from the Stripe webhook and page.js.
InayaProofRegistry	0xEdF431...CEBcB	Merkle-root storage per asset + chunk-proof verification (registerMerkleRoot, verifyChunkProof). Reads InayaCustody to confirm the caller owns the asset before letting them register a root.

> Also in contracts/ but not part of the six above: InayaStaking (Synthetix-style staking-rewards pool, 0/30/90-day lock tiers at 1.00x/1.25x/1.50x multiplier), InayaEgressTimelockVault (holds egress-fee $INAYA in 180-day epochs, forwards it to Staking; also sweeps USDT storage revenue to the operational treasury), InayaCorporateEscrow (drips a node operator's 39% COGS share of a corporate invoice over 12 monthly releases), and MockINAYA (test-only ERC-20 stand-in for $INAYA, not deployed to testnet under the real token address). All four are deployed via their own scripts/deploy-*.cjs; addresses live in .env.local and are read by page.js as stakingContractAddress / etc. — not restated here to avoid publishing addresses that weren't independently re-verified for this document.

Security Layer (4 contracts)

Decentralized threat intelligence — fully separate dependency graph from the core protocol above. Full detail in Section 09.

CONTRACT	ADDRESS	JOB
InayaThreatRegistry	0xb374dE...8adD	Threat status ledger, updated only by InayaThreatReporter.
InayaThreatReporter	0xe22EbA...6450B	confirmThreat() — relayer-only, called once a threat crosses the confidence threshold off-chain. Writes into ThreatRegistry.
InayaNodeReputation	0xD25836...9E27A	Periodic on-chain checkpoints of node reputation scores (real-time reputation itself lives off-chain).
InayaSecurityPolicy	0xCE1646...1976a	Versioned policy hash + URI, so a client can verify a cached policy offline without a live RPC call.

Trust-model honesty, on the record. InayaNodeRegistry's own header comment is explicit: node metrics are "coordinator-verified," not cryptographically proven — an authorized verifier wallet attests uptime/capacity off-chain. InayaProofRegistry's chunk-proof verification is currently onlyOwner (the backend checks proofs itself); the contract's own comment lays out a documented future path to make this permissionless and stake-slashing-backed. Neither is hidden — both are stated in the contract source itself.

SECTION 05

Client-Side Encryption & Storage — the custody-sdk

"InayaKernel" (custody-sdk) is the code that actually makes the zero-knowledge claim true. Published as @inaya-network/custody-sdk, used identically by the web dApp and the mobile app.

1. **Key derivation.** deriveVaultKey() runs PBKDF2 (default SHA-256, 100,000 iterations) over the user's passkey plus a fresh random 16-byte salt, producing a 32-byte AES key. The passkey itself never leaves the device and is never transmitted anywhere.
2. **Encrypt + shard.** disperseAndSlice() reads the file, base64-encodes it, AES-GCM-256 encrypts it with a fresh random IV, prepends salt+IV to the ciphertext, then splits that resulting base64 string at its exact character midpoint into two shards (shardAlpha / shardBeta). This is literal binary bisection — neither half means anything on its own, but it is not erasure coding or Reed-Solomon.
3. **Pin to IPFS.** The SDK itself does NOT upload anywhere — that's the calling app's job. page.js pins each shard to Pinata separately and gets back two CIDs (cidAlpha, cidBeta).
4. **Anchor on-chain.** anchorToLedger() calls InayaCustody.batchRegisterAssets(fileHashes, fileSizes, shardACIDs, shardBCIDs) — the CIDs (pointers, not the shard bytes themselves) are what actually gets written on-chain.
5. **Reconstruct + decrypt.** retrieveAndReconstruct() reads InayaCustody.assets(fileHash) to get both CIDs, fetches both shards from the Pinata gateway in parallel, concatenates them back into the original base64 string, splits out salt/IV/ciphertext, re-derives the AES key from the user's passkey, and decrypts.

> **Zero-knowledge, verified against the actual code — not assumed.** Every encryption, key-derivation, and decryption step happens locally using only inputs the device already has (passkey, file bytes). No network call anywhere in custody-sdk's source carries plaintext or the derived key. The only backend/chain-touching write is a file hash, size, and two CID strings — never file content, never the key.

A second, separate feature — sharing without exchanging the raw passkey:

- encryptForPublicKey / decryptWithSecretKey / deriveEncryptionKeypairFromSignature — re-wraps the passkey itself (not the file) using X25519 ECDH + HKDF-SHA256 + XChaCha20-Poly1305 sealed-box encryption, so a recipient can unlock a shared vault entry without the passkey ever being transmitted in the clear.

> Known gap worth flagging: the SDK's own bundled example (examples/node-script.mjs) skips the Pinata step and passes raw ciphertext directly into anchorToLedger's CID parameters — it only "works" because the example never calls retrieveAndReconstruct against real IPFS. The production code path in page.js does the Pinata step correctly; the example just isn't a faithful end-to-end demo.

Release verification — the SDK doesn't have to be trusted blindly.

Verifiable Inaya Client SOW (September 2026): as of custody-sdk v1.0.10-beta, every tagged release is independently reproducible and verifiable, not just claimed open-source. Full mechanism and third-party verification commands: custody-sdk/docs/VERIFYING_RELEASES.md.

1. **Reproducible build.** custody-sdk ships with no build step — src/*.js is exactly what's in git (ESM, explicit .js import extensions, hand-written .d.ts). "Reproducible" reduces to "the published tarball is provably the tagged source, file for file," not a compiled-artifact-determinism problem.

2. **Published hash — and a real bug found and fixed computing it.** Every tagged release (.github/workflows/release.yml) publishes a git-tree-hash and an npm-tarball SHA-256 to CHECKSUMS.md and the GitHub Release. First implementation used git archive HEAD | sha256sum — verification then found this isn't reproducible across git versions (confirmed directly: local git 2.55.0 and ubuntu-latest's git produced different hashes for the identical tagged v1.0.9-beta commit, which would make an honest verifier see a false mismatch). Fixed in v1.0.10-beta by publishing git rev-parse <tag>^{tree} instead — git's own content-addressed tree object id, identical on every git install by construction.

3. **Independent verification, actually performed.** Not just documented — run: v1.0.10-beta's git-tree-hash was independently recomputed on a separate machine and matched CHECKSUMS.md; the GitHub Release tarball was downloaded fresh and its SHA-256 matched; every file inside it was diffed byte-for-byte against the tagged git source and found identical. npm provenance (npm publish --provenance) separately attaches a signed, publicly-checkable statement of exactly which CI run produced the package (npm audit signatures).

4. **Content-addressed delivery.** The release tarball is also pinned to IPFS (Pinata) at publish time; the resulting directory CID is recorded in CHECKSUMS.md and the GitHub Release, IPFS pinning kept non-blocking (continue-on-error) so a Pinata-side outage can't prevent npm publish. Fetching by CID — not from any server Inaya controls — and hashing what comes back is a second, independent confirmation of the exact artifact.

> **What this does and does not prove.** Guarantees: the code that produced a release is exactly the tagged source; anyone can independently re-derive the published hashes; as of the same release, the web dApp (src/lib/clientCrypto.js, src/app/page.js) and inaya-mobile both call this published npm package directly rather than each running its own separate crypto — verified by a committed cross-implementation test, custody-sdk/test/webCryptoCompat.test.mjs, proving the SDK's @noble-based AES-GCM/PBKDF2 is byte-identical to and cross-decryptable with the dApp's former inline crypto.subtle implementation. Does not guarantee: that inayanetwork.com is serving this exact build at this exact moment, or an implementation-bug-free audit of the primitives beyond that cross-compatibility test.

SECTION 06

Node Operator Layer

What someone actually runs to operate an Inaya storage node today, and what it does and does not do.

NODE-DAEMON (PUBLISHED NPM CLI)

Commands: login, register <capacityGB>, start, report <indicator> (Security Layer), service install/uninstall. Wallet key is PBKDF2+AES-GCM encrypted at rest locally (~/.inaya/node-daemon/config.json). register() calls InayaNodeRegistry.registerNode() on-chain AND separately POSTs to the backend's /api/nodes/register for capacity bookkeeping. start() runs a 5-minute heartbeat loop POSTing telemetry to /api/nodes/heartbeat.

WHAT IT DELIBERATELY DOES NOT DO

It does not store or serve shards, does not host any file content, and does not execute settlement/payout logic — the daemon's own code comments are explicit that commission settlement is verifier- and relayer-driven, not something the daemon touches. Its on-chain surface is exactly two calls: registerNode and a read of nodes(address). It is an identity + registration + heartbeat agent, nothing more, today.

Settlement flow (InayaNodeRegistry, off-daemon):

- An authorized verifier wallet calls queueSettlement / queueSettlementsBatch — computes commission by tier, does not move funds yet.
- After a mandatory 36-hour delay, releaseSettlement is publicly callable by anyone (deliberately — a single compromised verifier key can queue a bad settlement but can never immediately drain funds).
- Corporate invoice revenue flows separately: RevenueRouter.processCorporateInvoice() → InayaCorporateEscrow.createEscrow() escrows a node operator's 39% COGS share and drips it out over 12 monthly releases via releaseMonthlyPayout(), callable by anyone once due.

SECTION 07

Web dApp — Protocol Surface

src/app/page.js — one large tabbed SPA. Every tab below is wallet-driven except Referrals (email + KYC only, no wallet).

TAB	WHAT IT DOES
Network Home	Landing panel only — status tiles, CTA into Sovereign Vault, cross-promo into Business Workspace / desktop downloads. No contract or API calls of its own.
Faucet	POSTs to /api/faucet; the route dispenses test $INAYA/USDT server-side from a custodial faucet wallet. Self-limits by checking the wallet's existing balance rather than a visible cooldown timer.
Sovereign Vault	The core product. Upload: encrypt+shard client-side (custody-sdk) → pin to Pinata → InayaCustody.batchRegisterAssets() → InayaProofRegistry.registerMerkleRoot() per file. Download: reverse of the same path. Also has a no-wallet path for Stripe card customers (upload happens locally, on-chain registration happens server-side post-payment).
Corporate Reserve (sidebar panel + Business Model tab)	The sidebar tier selector (250/500/1000 TB) only sets display state and enforces per-plan upload size limits — it is NOT the purchase flow. The real purchase lives on the Business Model tab: approve USDT → RevenueRouter.processCorporateInvoice() → approve USDT → InayaCorporateEscrow.createEscrow() escrowing the 39% operator COGS share over 12 months.
Staking	Reads InayaStaking for APY/tier display. Stake (with 0/30/90-day lock tier selection, real on-chain lockExpiry), Unstake (blocked until lock expires), Claim reward — direct calls to stake()/withdraw()/claimReward().
My Dashboard	Read-only aggregation of state already fetched elsewhere — PAYG transaction history with BscScan links, Corporate Reserve status, staking overview. No new contract calls.
Referrals (ReferralSection.js)	Deliberately no wallet anywhere — email + Didit KYC only, specifically to detect self-referral. Real KYC gate: referral credit only counts once Didit verification resolves.
Genesis Airdrop	Two mechanics, neither with an on-chain claim function today: an "Upload Reward" progress-bar estimate (0.01 $INAYA per upload, capped at 0.3/wallet, display-only, not minted), and a "Contributor Allocation" (Community/Developer/Moderator buckets of a 1,000,000-token pool) that routes to Google Forms for manual review — real distribution is off-app at TGE, not automated by this code.

SECTION 08

Business Workspace — SaaS Layer

A separate, structurally distinct product built on the same infrastructure: companies get organizations, departments, projects, and documents with real server-enforced permissions — sold and priced independently of storage/token economics, no wallet literacy required.

- Auth: email magic-link OR Google Sign-In, session cookie based (org.js — createSession/consumeLoginToken), completely separate from the wallet-signed-message pattern used elsewhere.
- Hierarchy: Org → Department → Project → Document, each level with its own role/permission scoping (canManageOrg gates admin actions; VIEW/EDIT/MANAGE per document).
- Workflow: documents move through a real state machine (DRAFT → PENDING → etc.) with an atomic transition function and a persisted activity log — not just a status field that gets overwritten.
- Sharing: explicit per-person grants (document-permissions.js) plus tokenized public share links (hash-and-compare, atomic single-consume where applicable) — same pattern conventions as the Data Room's magic-link/session tokens, implemented independently for each feature.
- Storage model: documents here are encrypted client-side before upload using the same passkey-based flow as the Sovereign Vault, then pinned as JSON-shard blobs — NOT the same storage path as the Data Room, which stores plain, unencrypted PDFs for inline viewing (see Section 11 for why that's a deliberate, different choice).
- Billing: Stripe subscription plans (orgPlans.js), seat limits enforced server-side on invite, storage-size limits enforced server-side on document upload.
- AI: a permission-aware Business Assistant (Section 12) that only ever sees data the requesting user is actually authorized to see.
- Beyond documents: the workspace now covers real business operations on the same org/department/permission foundation — Tasks, CRM, Procurement, Inventory (Section 23), and a Finance & HR layer (Section 24, testnet demonstration scope). None of this required touching the base document/permission model; every module is additive.

SECTION 09

Security Layer — Decentralized Threat Intelligence

A separate DePIN-style subsystem: independent nodes report suspicious domains/IPs; the backend reputation-weights reporters and, once confidence crosses a threshold, anchors a CONFIRMED verdict on-chain. Marketed externally as "Inaya Firewall."

1. **Report.** A node (mobile/desktop client or node-daemon's report command) POSTs a signed observation to /api/security/report. Rate-limited (200/day/node), upserted per node+threat+day.
2. **Aggregate.** computeThreatConfidence() reputation-weights every independent reporter over a 14-day window. Node reputation is real-time off-chain, only checkpointed on-chain periodically — confirmed reports raise it, false positives cost 3x as much (asymmetric penalty by design).
3. **Confirm.** Crossing CONFIRM_THRESHOLD_BPS (75%) triggers exactly one relayer-signed InayaThreatReporter.confirmThreat() call. A single new/low-reputation node can never force a confirmation alone — verified live in testing: 4 neutral-reputation nodes capped out at 52% confidence, well short of threshold.
4. **Surface.** Public /security web page (flagship AI-assistant card, destination checker, live threat feed, Inaya Firewall banner), mobile SecurityScreen (Monitor/Protect/Strict modes, local allow/blocklist, network overview), desktop enforcement (Windows netsh advfirewall / Linux iptables block, IP-literal only — not run against a real machine yet, reviewed line-by-line only), admin dashboard, and a dedicated Security AI Assistant that is explicitly forbidden from inventing evidence — every answer is grounded only in verified security_* collection data.

> Plaintext indicators (domains/IPs) never touch the chain — threat IDs are keccak256 of the normalized indicator; the backend holds the plaintext↔hash mapping (src/lib/security.js).

SECTION 10

Inaya Learn — Educational Platform

A YouTube-based educational discovery layer inside both the web dApp and mobile app — turns Inaya into a daily-use destination beyond storage/staking.

- Search/browse via the real YouTube Data API v3, backend-cached (MongoDB TTL index) specifically because search.list costs 100 quota units/call against a 10,000/day default quota — caching is load-bearing infrastructure here, not an optimization.
- Categories and curated collections (Learn Web3 / Learn AI / Learn Programming) are a hardcoded, git-deployed config file, not an admin-editable database — deliberate V1 scope cut.
- Local-first save/progress (AsyncStorage on mobile, localStorage on web), optional backend sync once a wallet connects.
- An AI Tutor grounded in the opposite guardrail philosophy from the Security Assistant: it teaches using its own general knowledge, only calling tools to check the user's own saved videos/progress — never gated to a narrow data set.

SECTION 11

Investor Data Room

A dedicated, branded document room for sharing investor materials with per-visitor view tracking — replaces sharing a Google Drive folder blind.

- Single shareable link, email required (not optional) + NDA click-through, per-visitor magic-link email verification.
- Deliberately different storage model from the Business Workspace: documents here are plain, unencrypted PDFs (server-proxied streaming, never a raw Pinata URL exposed) so they can be viewed inline with real duration tracking — the Business Workspace's client-side-encrypted-blob model is the wrong fit when the entire point is casual inline viewing with analytics.
- View events (open + 15-second heartbeat + close) roll up into a per-visitor engagement table for the admin — exactly who looked at what, for how long.

SECTION 12

The AI Assistants

Four assistants — Docs, Business, Security, and Learn Tutor — sharing one architectural pattern (Gemini function-calling, 5-round tool loop, 429/503 retry with backoff) but deliberately different guardrail philosophies. Three of the four (Docs, Security, Learn) are now also grounded by a shared RAG retrieval layer — see Section 25 for how that actually works; this section stays focused on what each assistant is FOR.

ASSISTANT	WHERE	GUARDRAIL PHILOSOPHY
Docs Assistant	Public site-wide chat widget	Grounded in the project's own docs/FAQ/fundraising-doc corpus via RAG retrieval (Section 25) instead of a static hardcoded knowledge block — answers cite which source they came from.
Business Assistant	Business Workspace sidebar	Permission-aware — every tool call re-checks the requesting user's actual document/project/finance/HR access; never surfaces data they couldn't otherwise see. Uses direct permission-scoped MongoDB tool-calling, not RAG — this is live, per-org private data, never appropriate to put in a shared vector index.
Security Assistant	Public /security page + mobile SecurityScreen	Strict evidence-grounding — explicitly forbidden from inventing or generalizing beyond verified security_* collection data (live tool-calling) or the RAG-indexed security policy/explainer docs (Section 25). "Explain phishing in general" is fine; fabricating a specific threat's status is not.
Learn AI Tutor	Web + mobile Learn section	The opposite on purpose — teaches using its own general knowledge like a real tutor; tools (including RAG search over the current video's transcript, Section 25) exist only to ground it in the user's own saved videos/progress, never to gate what subject matter it can discuss.

SECTION 13

Mobile App

inaya-mobile (Expo/React Native) is a superset of the web dApp's consumer features, plus Business Workspace, Learn, and Security — not a stripped-down companion app.

- Wallet: MetaMask Connect Multichain (direct deeplink to MetaMask Mobile), configured for BNB Chain Testnet specifically.
- Business Workspace: full org/department/project/document hierarchy, permissions, sharing, Bearer-token session auth (separate from web's cookie session), biometric app-lock, Google Sign-In.
- Security: SecurityScreen with protection mode, destination checker, local allow/blocklist, network overview, AI Security Assistant chat.
- Learn: search/browse/watch/save/progress via react-native-youtube-iframe, plus a global AiTutorButton (header icon → full-screen modal) reachable from every Learn screen.
- Watcher Pioneer, Referrals, Faucet-adjacent flows, and Genesis Airdrop all mirror the web dApp's mechanics 1:1 against the same backend routes.

SECTION 14

Desktop Apps

Two separate Tauri (Rust) apps — deliberately thin wrappers, no bundled frontend. Each app's dist/ is a blank placeholder HTML page; the Rust shell just points its webview at the live production URL and layers native OS chrome (tray, menu, notifications, auto-update) on top.

APP	WRAPS	NATIVE ADDITIONS BEYOND THE WEB PAGE
inaya-desktop ("Business Workspace")	inayanetwork.com/business	System tray/minimize-to-tray, native menu, native Windows toast for pending approvals (polls /api/orgs/pending-approvals), signed auto-updater, real firewall-block enforcement (netsh/iptables) for the Security Layer.
inaya-dapp-desktop ("Inaya Network")	inayanetwork.com/	Same tray/menu/updater pattern, no notifications plugin (no equivalent concept on the main dApp). Detects window.__TAURI__ and leads with WalletConnect instead of extension-based wallets, since a native webview has no browser extensions.

> Chosen over Electron after measuring both: 8.3MB installed vs 269MB for an equivalent Electron build, since Tauri rides the OS's own WebView2 (Windows) / WebKitGTK (Linux) instead of bundling Chromium. Not code-signed yet (deferred decision, see Section 20) — Windows installs show an expected SmartScreen warning.

SECTION 15

Backend Infrastructure & Conventions

One Next.js app (inaya-network-dapp) serves the web dApp, Business Workspace, every public page, and the entire API surface every client (mobile, both desktop apps) calls.

- Data layer: MongoDB, one lib file per feature domain (security.js, learn.js, dataroom.js, activity.js, orgs.js, watcherPioneer.js, feedback.js...) — each with the same shape: getXCollections(), ensureXIndexes() (module-level idempotency guard), validateXInput() functions that throw (fail-closed).
- Cron jobs (Vercel Cron, vercel.json): settlement-release (midnight), security reputation checkpoint (3am — moved off a 6-hourly schedule after it silently broke every deployment for weeks on the Hobby plan's once-daily cron limit).
- Two admin surfaces: /admin (Enterprise Dashboard — revenue, usage, DAU/WAU, feedback) and /admin/security and /admin/dataroom, all gated by the same ADMIN_DASHBOARD_SECRET ?key= pattern, not a full multi-admin auth system (solo-founder-appropriate, by design).
- Every route follows the same shape: export const dynamic = "force-dynamic", outer try/catch → 500 + console.error, inner try/catch → specific 4xx.

SECTION 16

Identity & Auth Patterns

PATTERN	USED BY	MECHANISM
Signed-message wallet auth	Sovereign Vault sign-up, Security Layer node reports, Watcher Pioneer	ethers.verifyMessage over a reconstructed message string, 5-minute freshness window — reimplemented locally per-feature rather than shared, by established convention in this codebase.
Cookie session (magic-link / Google)	Business Workspace (web)	orgs.js — generateToken/hashToken, TTL-indexed magic_links collection, consumeLoginToken.
Bearer token session	Business Workspace (mobile)	Same session concept as web, token-based instead of cookie-based since mobile has no cookie jar shared with a browser.
Anonymous device/visitor ID	Referrals, Watcher Pioneer, activity pings, Security destination checker, Learn progress (no wallet)	Client-generated UUID cached in localStorage/AsyncStorage, upgraded to wallet address once one connects — same trust model reused across every feature that needs to work pre-wallet.
Investor Data Room session	Data Room only	Its own magic-link + session-cookie pair, deliberately separate from Business Workspace's — different visitor identity model (name+email+NDA, not org membership).

SECTION 17

Activity & Analytics (DAU/WAU)

One small, shared primitive tracks daily/weekly active users across all three surfaces — dApp, Business Workspace, mobile.

- One MongoDB doc per identity+surface+day, upserted (not inserted) — repeated pings the same day are free no-ops, so dedup is structural, not query-time logic.
- DAU = distinct identities pinging today; WAU = distinct identities over the trailing 7 days — both computed with a plain distinct() query since the per-day collapse keeps the working set small.
- Identity: wallet address if connected, otherwise the same anonymous visitor/device ID pattern from Section 16.
- Surfaced as one more section on the existing /admin dashboard — not a new page, not a new auth system.

SECTION 18

End-to-End Flow: Uploading a File

1. **1.** User connects wallet, signs a verification message (Sovereign Vault sign-up), sets a master passkey, selects a file.
2. **2.** custody-sdk derives an AES key from the passkey (PBKDF2), encrypts the file (AES-GCM-256), and splits the ciphertext into two shards.
3. **3.** Each shard is pinned to Pinata/IPFS independently, returning two CIDs.
4. **4.** Client checks for a duplicate asset hash, reads live fee rates, approves USDT/$INAYA if needed, then calls InayaCustody.batchRegisterAssets() — this is the transaction that actually makes the upload "real."
5. **5.** Client separately calls InayaProofRegistry.registerMerkleRoot() per file (a Merkle root over the file's chunks) — a second, independent transaction; if this one fails, the custody registration from step 4 is not rolled back.
6. **6.** On download, the reverse: read InayaCustody.assets(fileHash) for both CIDs → fetch both shards from Pinata → concatenate → derive the same AES key from the user's passkey → decrypt. The passkey never leaves the device at any point in either direction.

SECTION 19

End-to-End Flow: A Threat Getting Confirmed

1. **1.** Independent nodes each observe the same suspicious domain/IP and POST a signed report to /api/security/report — rate-limited, deduped per node+threat+day.
2. **2.** computeThreatConfidence() reputation-weights every independent reporter within a 14-day lookback window.
3. **3.** Once aggregate confidence crosses 75%, the backend (acting as an authorized relayer) fires exactly one InayaThreatReporter.confirmThreat() call — anchoring category, confidence, and a hash of the contributing-node list on-chain.
4. **4.** The public /security page, mobile SecurityScreen, and desktop enforcement all read the same confirmed-threat feed from that point forward — a confirmed domain gets blocked at the app layer everywhere, and at the OS firewall layer on desktop for a real, confirmed threat.

SECTION 20

Known Gaps & Deliberate Deferrals

Stated plainly, not buried — these are conscious scope cuts, not things quietly forgotten:

- Node-daemon does not store/serve shards or run settlement logic — it's identity + registration + heartbeat only, today.
- InayaEgressTimelockVault has no code path yet that actually pays INAYA/USDT into it — it only accrues balance once something is wired to send it.
- InayaProofRegistry's chunk-proof verification is onlyOwner (backend-checked) rather than permissionless/stake-slashing-backed — a documented future path, not built yet.
- Real Android VPN/packet interception for the Security Layer is out of scope — mobile enforcement checks destinations inside the Inaya app only.
- Desktop firewall-block commands (netsh/iptables) are written and reviewed but have not been executed against a real machine.
- Neither desktop app is code-signed (Authenticode) — investigated, no free option exists for closed-source software, deferred to mainnet rather than spend on it now.
- Genesis Airdrop has no on-chain claim function — both mechanics are display/manual-review only, real distribution happens off-app at TGE.
- macOS is out of scope for both desktop apps, per Apple Developer Program cost + notarization constraint.
- Finance & HR (Section 24) is explicitly a testnet demonstration/validation layer — no PDF invoice generation, no multi-currency conversion, no regulated banking/payment-processor integration, no payroll/tax processing.
- RAG (Section 25) deliberately never ingests custody-sdk's own docs or the legacy KNOWLEDGE_ARTICLES collection, and never ingests private per-org Business Workspace data or per-identity Security events into the shared vector index — those stay live, permission-scoped tool calls, by design.

SECTION 21

Oracle & Automation Layer

A third, independent subsystem alongside the core protocol and Security Layer: an on-chain registry of approved external data sources, an adapter that validates every submission before trusting it, and an off-chain keeper that executes pre-approved contract actions under smart-contract rules — never arbitrary admin commands. Deployed and running live on BSC Testnet, publicly verifiable at inayanetwork.com/automation.

CONTRACT	ADDRESS	JOB
InayaOracleRegistry	0x0b4695...FdD90	Owner-approved data sources: which address may submit for which data type, active/inactive, emergency disable. Holds no data itself.
InayaOracleAdapter	0x44E9E1...4789	The actual data store other contracts read. Every submission is validated on-chain — not from the future, not already stale, not faster than the source's minimum interval, not an outlier beyond a configurable max deviation from the previous value. A submission failing any check reverts; nothing partially-invalid is ever recorded.
InayaAutomationRegistry	0xa24Eae...ADf53	A transparent record of approved automated tasks and what they've actually done — deliberately holds no special calling rights over any target contract and never forwards a call. The off-chain worker calls target functions directly, under whatever access control they already enforce.

1. **Real data, not a simulated demo.** The oracle source live today is the INAYA/USDT spot price, read directly from the live PancakeSwap testnet pool's reserves — the same price computation the egress checkout flow already uses. Not a fabricated number for demonstration purposes.
2. **Real automation target, not a toy example.** InayaNodeRegistry's releaseSettlementsBatch() was already permissionless and time-locked — anyone could call it once a settlement's 36-hour delay passed, but nothing was calling it automatically. The keeper now does, on a schedule, running against a genuine existing function rather than one built just for this demo. The first real run found and released an actual previously-unclaimed settlement.
3. **Fails safe, not silently.** If oracle data goes stale, dependent automation skips that pass rather than acting on unverified data — proven live by deliberately lowering the staleness threshold and confirming the system correctly detects and reports it, then restoring normal operation.

> The keeper is a standalone script an operator runs (manually or on their own schedule) with their own key — not a hosted service with standing infrastructure access. Its on-chain authority is narrow by construction: it can only submit to oracle sources it's explicitly registered for, and it can only call functions that are already safe to call permissionlessly.

SECTION 22

Where Everything Lives

AREA	PATH
Core protocol contracts	contracts/ (repo root) — InayaStaking.sol, InayaEgressTimelockVault.sol, InayaCorporateEscrow.sol, InayaNodeRegistry.sol, InayaProofRegistry.sol, MockINAYA.sol
Security Layer contracts	contracts/InayaThreat{Registry,Reporter}.sol, InayaNodeReputation.sol, InayaSecurityPolicy.sol
Oracle & Automation contracts	contracts/oracle/InayaOracleRegistry.sol, InayaOracleAdapter.sol, contracts/automation/InayaAutomationRegistry.sol
Deploy scripts	scripts/deploy.js, deploy-staking.cjs, deploy-vault.cjs, deploy-escrow.cjs, deploy-node-registry.cjs, deploy-verifier-safe.cjs, deploy-security-layer.js, deploy-oracle-automation.js
Automation keeper + tests	scripts/automation-worker.mjs, scripts/test-automation-worker.mjs, test/OracleAutomation.test.js
Client-side crypto SDK	inaya-network-dapp/custody-sdk/src/ (crypto.js, index.js, contracts.js)
Node operator CLI	inaya-network-dapp/custody-sdk/packages/node-daemon/
Release verification (reproducible build, published hashes, IPFS pin)	custody-sdk/.github/workflows/release.yml, CHECKSUMS.md, docs/VERIFYING_RELEASES.md, scripts/pin-release.mjs
Cross-implementation crypto compatibility proof	custody-sdk/test/webCryptoCompat.test.mjs
Backend + web dApp + Business Workspace	inaya-network-dapp/src/app/, src/lib/, src/app/api/
Business Operations (Tasks/CRM/Procurement/Inventory)	src/lib/{task,deal,purchase-request,purchase-order}-workflow.js, inventory.js; src/app/api/orgs/{tasks,crm,procurement,inventory}/
Finance & HR	src/lib/{invoice,expense,employee,leave}-workflow.js, attachments.js; src/app/api/orgs/{finance,hr}/
RAG infrastructure	src/lib/rag/ (chunking, embeddings, vectorStore, retrieve, ingest); src/app/api/admin/rag/, src/app/api/cron/rag-reingest/
Mobile app	inaya-mobile/src/
Desktop — Business Workspace wrapper	inaya-desktop/src-tauri/
Desktop — dApp wrapper	inaya-dapp-desktop/src-tauri/

SECTION 23

Business Operations — Tasks, CRM, Procurement, Inventory

Four modules added to the Business Workspace on the exact same org/department/permission foundation Section 08 describes — no new auth system, no new storage system, every record department-scoped through the same canAccessDepartment gate every document already goes through.

MODULE	CORE OBJECTS	NOTABLE MECHANIC
Tasks	tasks (assignee, dueDate, status)	Real state-machine workflow, not just a status field — the {from,to,requiresManage,activityAction} transition-table pattern this whole layer reuses everywhere.
CRM	crm_contacts, crm_deals	A contact IS the unified Lead/Customer record — type flips LEAD→CUSTOMER on conversion rather than the record being recreated, so history/notes/deals stay attached.
Procurement	purchase_requests, purchase_orders, suppliers	PR→PO approval chain; PO receiving genuinely moves real Inventory stock — the two modules are integrated, not just adjacent.
Inventory	products, warehouses, stock_levels, stock_movements	stock_levels is a materialized view, only ever changed via $inc; stock_movements is the real append-only ledger — same 'ledger is truth' discipline used elsewhere in this codebase (faucet lifetime-caps, Finance's leave balances).

Shared infrastructure, not per-module reinvention:

- org_activity — one domain-generic audit log every module's transitions write into, not a bespoke activity table per module.
- getAccessibleScope() (document-permissions.js) — the one function that resolves 'everything this member can see across the whole org,' extended incrementally as each module shipped rather than four separate resolvers.
- AI tools: list_tasks, list_contacts/list_deals, list_suppliers/list_purchase_orders, list_products — all permission-scoped through the same getAccessibleScope() the human-facing routes use, so the Business Assistant never sees more than the requesting user could see themselves.

> Full detail (data model, workflow tables, permission model, test coverage, explicit out-of-scope list) in BUSINESS_OPERATIONS_TASKS.md, _CRM.md, _PROCUREMENT.md, and _INVENTORY.md at the repo root.

SECTION 24

Finance & HR Layer

Invoices, Expenses, Payments, and CSV reporting (Finance); Employee records, Employee documents, Leave management, and Department Administration (HR) — a testnet demonstration/validation layer, explicitly not regulated banking, tax filing, or payroll processing. Every Finance/HR screen carries a visible 'Testnet / Beta' badge.

Roles are additive, not a restructure.

org_members.role stays a single fixed string (owner|admin|member), checked in exactly one place (canManageOrg). Finance and HR each get their own new, OPTIONAL fields on the same document instead: financeRole (null|manager|staff), hrRole (null|manager|staff), managedDepartmentIds (new — Department Manager, a role that didn't exist before this layer). Zero risk to any existing module's gates.

CONCEPT	MECHANIC
Invoice lifecycle	DRAFT→SENT→PAID/CANCELLED, plus a cron-driven SENT→OVERDUE (invoices-mark-overdue, nightly, CRON_SECRET-gated) — the one transition in this entire layer that isn't user-invoked. Idempotent: a PAID invoice with a past due date is never touched, because the cron filters on status, not on date.
Expense approval	submit→PENDING_APPROVAL→APPROVED/REJECTED, approve/reject gated by canManageFinance (Finance Manager or org owner/admin) — Finance Staff can submit but not approve their own or anyone else's.
Employee identity	employees.memberEmail is nullable — set when the employee has real workspace login, absent when HR is just tracking someone who never logs in. Either way is a valid record.
The 'Employee' role	Not a role string at all — any member viewing the employees record whose memberEmail matches their own session gets read access to that one record and their own leave requests. A data-scoping rule, not an enum value.
Leave balance	Computed fresh on every read (allocationDays − this year's APPROVED day-spans), never a mutable stored counter — same 'ledger is truth' discipline as Inventory's stock levels (Section 23).
HR/expense documents	A new attachments collection, deliberately NOT a reuse of org_documents (which mandates a projectId and a department/project permission model wrong for 'who can see this employee's contract') — reuses the same client-side encrypt/shard/pin pipeline, just records metadata differently.

A real bug this layer's own tests caught. getAccessibleScope() originally filtered a Department Manager's managedDepartmentIds against their own already-visible departments before querying, intended as a redundant-query optimization — but department-level visibility doesn't imply HR/employee visibility, which is gated separately. A Department Manager whose own membership already included their managed department (the normal case) ended up with an empty managedDeptIds and silently lost their HR grant. Caught by test/hr-workflow.test.mjs's Department Manager scope test, fixed by removing the incorrect filter — a genuine finding from testing against real Atlas, not a hypothetical.

Full detail in BUSINESS_OPERATIONS_FINANCE.md and HR.md at the repo root.

SECTION 25

RAG — Retrieval-Augmented Generation Infrastructure

The shared retrieval layer behind the Docs, Security, and Learn assistants (Section 12) — replaces the Docs Assistant's old static hardcoded knowledge block with real semantic + keyword search over a live, re-ingestable content index. Built on this project's real MongoDB Atlas cluster, not a bolted-on vector-DB vendor.

1. **Chunk & embed.** Static sources (docs pages, FAQs, the fundraising-docs content files, the security policy explainer, Learn's curated config, and lazily-ingested YouTube video transcripts) get chunked by heading/paragraph/structured-section as appropriate, then embedded via Gemini's gemini-embedding-001 (768 dimensions), content-hash-cached so re-ingesting unchanged content costs nothing.
2. **Index — native Atlas, both kinds.** rag_chunks carries both an Atlas Vector Search index ($vectorSearch) and an Atlas Search index ($search), both created idempotently in code (collection.createSearchIndex()) — no manual Atlas dashboard step required.
3. **Hybrid retrieve.** retrieveContext() merges vector + keyword results via Reciprocal Rank Fusion in application code (not a native $rankFusion stage, for broader Atlas-tier compatibility), then gates on a relevance threshold calibrated against real measurements on this project's own cluster — DEFAULT_MIN_RELEVANCE = 0.8, set after live-measuring relevant queries scoring ~0.89-0.91 cosine similarity against irrelevant ones at ~0.72-0.77 (a genuinely higher baseline than intuition suggested, and a real bug caught: an earlier 0.55 threshold plus a keyword-match override let an irrelevant query pass).
4. **Attribute the answer.** Every assistant response using RAG-retrieved context appends a formatted attribution of which source(s) it drew from — never a bare answer with no traceable origin.

Permission-safe by construction:

- Nothing private is ever ingested into the shared rag_chunks index — Security's per-identity events and Learn's per-wallet progress stay live, permission-scoped MongoDB tool calls (Section 12/23's pattern), never embedded text.
- custody-sdk's own docs and the legacy KNOWLEDGE_ARTICLES collection are deliberately excluded from ingestion — documented scope trims, not oversights.

Ops:

- Admin UI at /admin/rag (reingest, stats) plus a nightly cron (rag-reingest, CRON_SECRET-gated, same pattern as every other cron in this codebase).
- rag_query_log and rag_ingestion_runs give an honest record of what was actually retrieved and when content was last (re)indexed — metrics recording is fail-open, never blocks the caller.

> Full detail in RAG_INFRASTRUCTURE.md at the repo root.